

Student 1 Name: Arezo Nawrozi

- ID: 242102
- AUW Email: auw242102@auw.edu.bd

Student 2 Name: Zuhra Salangi

- ID: 242266
- AUW Email: auw242266@auw.edu.bd
-

1. Abstract

This project presents a C-based Student Grade Management System designed to automate academic record keeping, evaluate performance, and generate persistent output logs. Built using fundamental C programming concepts and the CS50 library, the application collects student details, dynamically assigns letter grades based on scores, recursively computes class totals, orders student ranks using a descending Bubble Sort algorithm, performs linear name searches, and writes all processed records to an external text file (`students.txt`). This system provides an intuitive terminal interface while demonstrating key procedural programming paradigms including custom data structures, sorting, searching, recursion, and file input/output (I/O) management.

2. Introduction

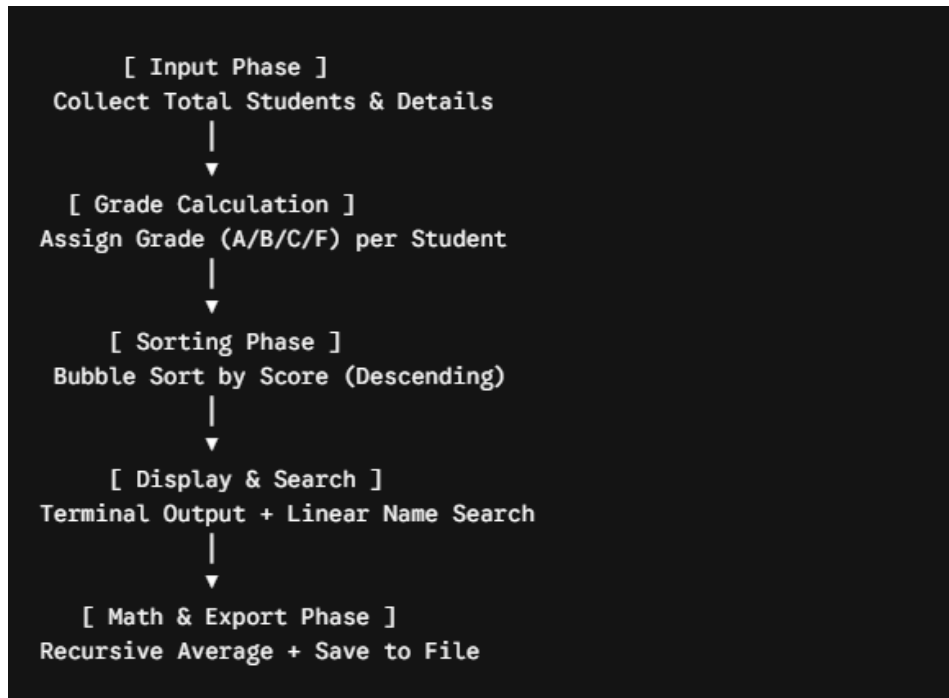
In academic environments, managing student performance data manually—such as recording marks, assigning letter grades, ranking students, and generating summary reports—is prone to human error and inefficiency. The objective of this project is to develop an integrated computational tool in C that automates these administrative tasks.

Key Project Objectives:

- Organize multi-attribute student data using custom composite types (`struct`).
- Implement automated letter grade assignments based on standard grading thresholds.
- Demonstrate algorithmic principles such as recursive arithmetic, sorting, and searching.
- Save generated records to disk (`students.txt`) to ensure data persistence across program executions.

3. Methodology / Implementation

The application is structured into modular functions, algorithm blocks, and standard data pipelines.



3.1 Data Structures

A custom composite structure named `Student` was defined using `typedef struct` to aggregate four attributes per record:

- `id (int)`: Unique numeric identifier.
- `name (string)`: Full name of the student.
- `score (float)`: Numerical mark (0–100).
- `grade (char)`: Letter grade ('A', 'B', 'C', or 'F').

3.2 Modular Functions

1. **`calculate_grade(float score)`**: Evaluates numerical scores and returns standard letter grade characters based on defined criteria ($\geq 90 \rightarrow \text{'A'}$, $\geq 80 \rightarrow \text{'B'}$, $\geq 70 \rightarrow \text{'C'}$, $< 70 \rightarrow \text{'F'}$).
2. **`sum_scores(Student students[], int count)`**: A recursive function designed to compute total class marks without explicit iteration loops. Its base case returns 0

when `count <= 0`; otherwise, it recursively adds `students[count - 1].score` to the call stack.

3.3 Core Processing Logic in `main()`

- **Input Loop:** Prompts the user for total headcount, creates a variable-length array `class_list[total]`, and collects individual field values using CS50 helper routines (`get_int`, `get_string`, `get_float`).
- **Bubble Sort:** Arranges student structs in descending order of their numerical scores using a nested double-loop strategy. Adjacent student scores are compared, and whole Student structures are swapped using a temporary buffer variable (`temp`).
- **Linear Search:** Requests a target student name from the user, iterates through `class_list`, and performs exact string matching via `strcmp()`. If a match is found, the loop exits early using `break`.
- **File Processing:** Uses standard file pointers (`FILE *file = fopen("students.txt", "w")`) to write all sorted records into disk storage using `fprintf()`.

4. Results / Output

4.1 Execution Flow & Screenshots

```
$ make Project
$ ./Project
How many students do you have? 6

--- Student 1 ---
Enter ID: 100
Enter Name: Arezo
Enter Score (0-100): 99

--- Student 2 ---
Enter ID: 101
Enter Name: Zuhra
Enter Score (0-100): 95

--- Student 3 ---
Enter ID: 102
Enter Name: Nargis
Enter Score (0-100): 70

--- Student 4 ---
Enter ID: 103
Enter Name: Alina
Enter Score (0-100): 80

--- Student 5 ---
Enter ID: 104
Enter Name: Mariam
Enter Score (0-100): 75

--- Student 6 ---
Enter ID: 105
Enter Name: Fatima
Enter Score (0-100): 85
```

```
=== ALL STUDENTS (Sorted by Score) ===
ID: 100 | Name: Arezo | Score: 99.0 | Grade: A
ID: 101 | Name: Zuhra | Score: 95.0 | Grade: A
ID: 105 | Name: Fatima | Score: 85.0 | Grade: B
ID: 103 | Name: Alina | Score: 80.0 | Grade: B
ID: 104 | Name: Mariam | Score: 75.0 | Grade: C
ID: 102 | Name: Nargis | Score: 70.0 | Grade: C

=== SEARCH FOR A STUDENT ===
Who do you want to find? Alina
Found them! Alina has score 80.0 and grade B

Class Average Score: 84.00

Saved records to 'students.txt' successfully!
$
```

4.2 Generated File Record (**students.txt**)

When executed with sample inputs, the file stream writes the following output directly to `students.txt`:

Plaintext

```
ID: 101, Name: Zuhra, Score: 92.5, Grade: A  
ID: 102, Name: Fatima, Score: 84.0, Grade: B  
ID: 103, Name: Mariam, Score: 68.5, Grade: F
```

5. Challenges and Limitations

1. **Lack of Input Validation:** The system assumes valid user inputs. If a user enters a score outside the 0–100 range or an invalid data type, the logic does not throw an explicit validation error.
2. **Case Sensitivity in Linear Search:** String matching via `strcmp()` is case-sensitive. Searching for "zuhra" will fail if the recorded name is "Zuhra".
3. **Volatility of Memory Arrays:** The program uses a stack-allocated variable-length array (`class_list[total]`). For large class sizes, this risks stack overflow compared to heap-allocated memory (`malloc`).
4. **File Overwrite Mode:** Opening the output file in "w" mode overwrites previous records rather than appending ("a") new entries to historical logs.

6. Conclusion and Discussion

The Student Grade Management System successfully integrates structural data grouping, custom functions, array manipulation, search/sort algorithms, recursive arithmetic, and persistent storage in C. By combining interactive I/O handling with procedural control structures, the project meets all functional requirements for systematic student performance evaluation and offline data logging.

7. Future Work

- **Dynamic Memory Allocation:** Refactor dynamic arrays using malloc, realloc, and free to allow runtime dynamic resizing and manage large datasets safely.
- **Database Integration & Binary Search:** Upgrade search efficiency to $O(\log n)$ using Binary Search on sorted fields and read existing data back from files upon initialization ("r" mode).
- **Robust Data Validation:** Implement checks ensuring scores remain strictly between 0.0 and 100.0 and add case-insensitive string searching algorithms.
- **Graphical User Interface (GUI):** Implement a graphical or web-based UI layer for user interaction.

8. References

1. Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
2. CS50 Library for C. Harvard University. CS50 Docs: <https://cs50.readthedocs.io/>

9. Individual Contributions

Team Member	Contributions & Responsibilities
Arezo	Designed the overall program architecture; defined the Student struct; implemented the primary control flow in main(); authored the linear search logic, class average computation, and file I/O handling (students.txt). Led report writing and visual documentation.
Zuhra	Implemented modular helper routines (calculate_grade logic and the recursive sum_scores function); developed and debugged the Bubble Sort algorithm logic for descending order; assisted in user input design and code testing.